UNIVERSIDADE FEDERAL DE ITAJUBÁ

SISTEMAS DE INFORMAÇÃO

Da Assistência à Agência: Uma Investigação Comparativa entre os Paradigmas Engenharia de Software 2.0 e Engenharia de Software 3.0

Isaac do Prado Almeida

Trabalho de Conclusão de Curso
Bacharelado em Sistemas de Informação



INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
UNIFEI - Itajubá

2026

do Prado Almeida, Isaac

Da Assistência à Agência: Uma Investigação Comparativa entre os Paradigmas Engenharia de Software 2.0 e Engenharia de Software 3.0 / Isaac do Prado Almeida. - Itajubá: UNIFEI, 2026.

11 p.

Trabalho de Conclusão de Curso (Sistemas de Informação) - Universidade Federal de Itajubá, Itajubá, 2026

Orientação: Isabela Neves Drummond

1. Agentes de IA. 2. Engenharia de Software. 3. Sistemas Multiagentes. 4. Orquestração de Agentes. 5. Large Language Models. I. Neves Drummond, Isabela, orient. II. Título.

FOLHA DE APROVAÇÃO

do Prado Almeida, Isaac. **Da Assistência à Agência: Uma Investigação Comparativa entre os Paradigmas Engenharia de Software 2.0 e Engenharia de Software 3.0**. 11 p. Trabalho de Conclusão de Curso (Sistemas de Informação), Universidade Federal de Itajubá, Itajubá, 2026.

Defesa em 23 de novembro de 2026.

BANCA EXAMINADORA:

Profa. Dra. Isabela Drummond

Prof. Dr. BBBB

Prof. Dr. CCCC

Prof. Dr. DDDD

Prof. Dr. EEEE

UNIVERSIDADE FEDERAL DE ITAJUBÁ
INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Da Assistência à Agência: Uma Investigação Comparativa entre os Paradigmas Engenharia de Software 2.0 e Engenharia de Software 3.0

Isaac do Prado Almeida

isaacpalmeida@unifei.edu.br

Orientadora: Isabela Neves Drummond

RESUMO

A Engenharia de Software tem passado por transformações paradigmáticas sucessivas ao longo de suas décadas de existência, e a adoção crescente de Inteligência Artificial generativa representa a mais recente e profunda dessas mudanças. A emergência dos Large Language Models (LLMs) introduziu um novo horizonte no desenvolvimento de software, inicialmente materializado por ferramentas copilotos que operam como assistentes centrados no desenvolvedor humano, caracterizando o que a literatura recente denomina Engenharia de Software 2.0 (ES 2.0). Paralelamente, pesquisas mais recentes descrevem um paradigma emergente denominado Engenharia de Software 3.0 (ES 3.0), no qual agentes de IA especializados colaboram autonomamente entre si ao longo de todo o ciclo de desenvolvimento de software. A transição entre esses paradigmas representa uma mudança estrutural no locus da agência dentro do processo de engenharia, deslocando o protagonismo de um desenvolvedor assistido por IA para sistemas de agentes que executam, raciocinam, planejam e validam de forma coordenada. Diante dessa evolução, este Trabalho Final de Graduação tem como objetivo investigar como a adoção do paradigma baseado em agentes impacta o processo de desenvolvimento de software em comparação ao paradigma predominante baseado em copilotos. Para viabilizar essa investigação de forma experimental e controlada, será desenvolvida a framework Uranium — um artefato de pesquisa construído com LangGraph e instrumentado com LangSmith — que servirá de infraestrutura para a execução de tarefas de desenvolvimento nos dois paradigmas. A pesquisa adota o método Design Science Research (DSR) como abordagem metodológica, combinando a construção de artefatos com a avaliação experimental por meio de estudo comparativo envolvendo desenvolvedores com diferentes níveis de senioridade e métricas quantitativas e qualitativas.

Palavras-chave: Agentes de IA; Engenharia de Software; Sistemas Multiagentes; Orquestração de Agentes ; Large Language Models

1 Introdução

A Engenharia de Software, desde sua formalização como disciplina científica na histórica conferência da NATO em 1968, tem sido marcada por ciclos contínuos de transformação. Cada um desses ciclos respondeu a desafios técnicos e organizacionais de sua época: da crise do software das décadas de 1960 e 1970 às metodologias ágeis dos anos 2000, da computação em nuvem ao DevOps, o campo sempre buscou formas mais eficientes, confiáveis e escaláveis de produzir sistemas de software. A chegada da Inteligência Artificial generativa ao cotidiano do desenvolvimento representa mais do que mais um instrumento nessa série de evoluções — ela redefine os papéis fundamentais dos atores envolvidos no processo.

A ascensão dos Large Language Models (LLMs), treinados sobre vastos corpora de texto e código, abriu possibilidades inéditas para o suporte ao desenvolvimento de software. Ferramentas como o Anysphere (2022) e Anthropic (2025), demonstraram que modelos de linguagem são capazes de compreender intenções expressas em linguagem natural e gerar código contextualmente relevante, acelerando tarefas de implementação, depuração e geração de testes. Esse conjunto de práticas — no qual um desenvolvedor humano utiliza assistentes de IA como copilotos, mantendo o controle decisório do processo — foi sistematizado pela literatura como Engenharia de Software 2.0 (ES 2.0), um paradigma de desenvolvimento centrado no engenheiro humano, que delega à IA funções de assistência e aceleração, mas reserva para si a condução do fluxo de trabalho.

Entretanto, as limitações intrínsecas das ferramentas copilotos tornaram-se progressivamente evidentes. Restrições nas janelas de contexto, a ausência de compreensão sistêmica dos fluxos de trabalho de engenharia, a falta de mecanismos integrados de garantia de qualidade e a sobrecarga cognitiva imposta ao desenvolvedor como responsável por toda a orquestração constituem barreiras ao aproveitamento pleno do potencial da IA no desenvolvimento de software. Esses limites motivaram uma evolução paradigmática: ao invés de um único assistente respondendo aos estímulos do desenvolvedor, sistemas compostos por múltiplos agentes especializados passaram a ser propostos como uma alternativa capaz de executar fases completas do ciclo de desenvolvimento de forma coordenada e autônoma.

Esse novo paradigma, denominado Engenharia de Soft-

© IMC-UNIFEI - Novembro de 2026

ware 3.0 (ES 3.0), foi formalmente proposto por **?** como uma abordagem centrada em intenção, na qual agentes de IA atuam como colaboradores inteligentes — denominados *AI Teammates* — que mantêm memória persistente, compreendem restrições arquiteturais e são capazes de gerar, testar e implantar artefatos de software de forma autônoma. Nesse paradigma, a intervenção humana é reservada para decisões estratégicas, revisão ética e validação final, enquanto o ciclo de desenvolvimento é conduzido por agentes especializados operando em coordenação. O surgimento de sistemas como MetaGPT, CodeAir e SDLC AutoPilot AI ilustra concretamente essa transição, demonstrando que arquiteturas multiagentes são capazes de entregar redução substancial de tempo de desenvolvimento mantendo ou melhorando indicadores de qualidade de código.

A comparação entre esses dois paradigmas constitui, portanto, um problema científico relevante e ainda insuficientemente explorado pela literatura. Embora existam avaliações de ferramentas individuais de ES 2.0 e demonstrações isoladas de capacidades de ES 3.0, a literatura carece de estudos comparativos controlados que confrontem os dois paradigmas em tarefas equivalentes, com controle rigoroso de variáveis e coleta sistemática de métricas. Compreender como a adoção de agentes autônomos especializados impacta dimensões como produtividade, autonomia, qualidade dos artefatos, rastreabilidade e custo é fundamental tanto para o avanço científico da área quanto para orientar decisões práticas de adoção por equipes de desenvolvimento.

Diante dessa lacuna, este trabalho tem como objetivo central investigar de que forma o paradigma ES 3.0, baseado em agentes de IA colaborativos, modifica o processo de desenvolvimento de software em comparação ao paradigma ES 2.0, baseado em copilotos assistivos. Para viabilizar essa investigação de forma experimental, será desenvolvida a framework **Uranium**, um artefato de pesquisa construído sobre LangGraph e instrumentado com LangSmith, cuja função é fornecer a infraestrutura necessária para executar tarefas de desenvolvimento no paradigma ES 3.0 de forma controlada e comparável ao paradigma ES 2.0. A Uranium não constitui o objeto de pesquisa em si, mas o meio pelo qual o estudo comparativo poderá ser conduzido com rigor metodológico.

A metodologia adotada é o Design Science Research (DSR), conforme sistematizado por Wieringa (2009, 2014), que oferece um arcabouço adequado para pesquisas que combinam a construção de artefatos com a geração de conhecimento científico por meio de avaliação empírica. O estudo comparativo será conduzido com aproximadamente cinco desenvolvedores de diferentes níveis de senioridade, utilizando o mesmo modelo de LLM em ambos os cenários, alterando exclusivamente o paradigma de desenvolvimento como variável de tratamento. As métricas coletadas abrangerão dimensões de produtividade, autonomia, qualidade dos artefatos, rastreabilidade e custo, permitindo análise quantitativa e qualitativa dos impactos observados.

2 Referencial Teórico

Este capítulo apresenta os principais conceitos que fundamentam esta pesquisa, abordando a evolução da Engenharia de Software impulsionada pela Inteligência Artificial, os paradigmas de Engenharia de Software 2.0 e Engenharia de Software 3.0, os fundamentos dos agentes inteligentes e sistemas multiagentes, bem como aspectos relacionados à orquestração, programação, segurança e governança de agentes de IA.

2.1 Engenharia de Software: Origens, Crises e Evolução dos Processos

A Engenharia de Software como disciplina científica nasceu de uma percepção de crise. Nas décadas de 1960 e 1970, o crescimento exponencial da complexidade dos sistemas de software evidenciou que os métodos artesanais e informais de programação então vigentes eram insuficientes para garantir qualidade, confiabilidade e prazos de entrega. A denominada "crise do software"era caracterizada por orçamentos estourados, prazos descumpridos, sistemas instáveis e projetos abandonados. A conferência da NATO em 1968 representou o marco formal de reconhecimento desse problema e lançou as bases para que o desenvolvimento de software fosse tratado com o rigor metodológico das demais engenharias.

A resposta inicial a essa crise foi a sistematização de processos de desenvolvimento. O modelo em cascata estabeleceu a sequencialidade das fases — análise, projeto, implementação, verificação e manutenção — como fundamento para a gestão de projetos de software. Embora representasse um avanço significativo em relação às práticas informais anteriores, este modelo apresentava limitações estruturais: sua rigidez diante de mudanças de requisitos e o longo intervalo entre a concepção e a entrega tornavam-no inadequado para ambientes dinâmicos.

As décadas seguintes testemunharam a proliferação de abordagens que buscavam superar essas limitações. A família de métodos ágeis, consolidada no **Manifesto Ágil** de 2001, representou uma ruptura paradigmática ao propor desenvolvimento iterativo e incremental, colaboração contínua com o cliente e capacidade de resposta a mudanças como valores centrais do processo. Práticas como Scrum, Extreme Programming (XP) e Kanban foram progressivamente adotadas pela indústria, transformando a dinâmica dos times de desenvolvimento. Em paralelo, o movimento DevOps emergiu como uma extensão da filosofia ágil ao domínio de operações, integrando desenvolvimento e implantação em pipelines contínuos de integração e entrega (CI/CD) e encurtando drasticamente os ciclos de feedback entre código e produção.

Cada um desses movimentos representou uma tentativa de lidar com a crescente complexidade dos sistemas de software mantendo qualidade e velocidade de entrega. Contudo, em todos esses paradigmas, a inteligência humana permanecia como o centro gravitacional do processo: era o desenvolvedor quem tomava as decisões, escrevia o código e interpretava os requisitos.

A introdução da Inteligência Artificial no processo de desenvolvimento mudaria esse cenário de forma progressiva, mas profunda. Inicialmente, ferramentas baseadas em análise estática e aprendizado de máquina foram incorporadas para detecção automática de defeitos, sugestão de refatorações e priorização de testes. Contudo, o salto qualitativo ocorreu com o surgimento dos Large Language Models (LLMs), modelos de linguagem de grande escala treinados sobre massivos corpora de texto e código, capazes de compreender e gerar linguagem natural e código-fonte com grau de qualidade anteriormente inalcançável. Esse salto inaugurou o que a literatura passou

a denominar ***AI-Assisted Software Development***, uma nova fase na qual a IA não apenas apoia tarefas isoladas, mas passa a ser uma participante ativa no processo de desenvolvimento.

2.2 A Evolução para Engenharia de Software 2.0 e 3.0

O conceito de Engenharia de Software 2.0 designa um paradigma de desenvolvimento no qual engenheiros humanos utilizam ferramentas de Inteligência Artificial generativa como copilotos assistivos. Nesse modelo, ferramentas como GitHub (2021), Anysphere (2022) e Anthropic (2025) aceleram tarefas de codificação, depuração e geração de testes, mas o fluxo decisório permanece centrado no desenvolvedor humano, que mantém controle sobre o processo e responsabilidade pelos artefatos produzidos. Estudos indicam que 75% dos desenvolvedores já utilizavam alguma ferramenta de IA assistiva em 2024, com previsão de 90% de adoção empresarial até 2028. Apesar dos ganhos de produtividade reportados, com reduções de até 45% no tempo inicial de codificação, a ES 2.0 introduz limitações sistêmicas que motivam a evolução paradigmática.

As limitações centrais da ES 2.0 são de três ordens. Em primeiro lugar, as restrições de janela de contexto dos LLMs, que tipicamente variam de 8.000 a 200.000 tokens, impõem que, ao desenvolver funcionalidades complexas abrangendo múltiplos arquivos, desenvolvedores frequentemente se deparam com perda de continuidade contextual, sendo forçados a reexplicar repetidamente convenções arquiteturais e decisões de design. Em segundo lugar, os assistentes copilotos operam como interfaces conversacionais sem compreensão inerente dos fluxos de trabalho de engenharia, criando uma fricção que o desenvolvedor deve superar manualmente. Em terceiro lugar, a garantia de qualidade permanece integralmente sob responsabilidade do desenvolvedor, com frequência negando os ganhos de produtividade prometidos pela IA assistiva.

Em resposta a essas limitações, **?** propuseram o conceito de Engenharia de Software 3.0, um paradigma centrado em intenção e orientado a agentes, no qual a IA assume o papel de colaborador inteligente — não apenas copiloto — capaz de compreender princípios de engenharia de software em profundidade. O modelo propõe um stack tecnológico conceitual composto por:

- Teammate.next, parceiros adaptativos com memória de longo prazo e compreensão arquitetural;
- IDE.next, interfaces orientadas a intenção que integram logs, diagramas e testes;
- Compiler.next, síntese de código com múltiplos objetivos incluindo custo, corretude e conformidade;
- Runtime.next, ambientes de execução cientes de SLA que supervisionam e validam mudanças geradas por agentes.

Nesse paradigma, a intervenção humana é reservada para decisões estratégicas, revisão ética e validação final.

A evidência empírica mais recente sustenta a relevância dessa transição. O estudo AIDev, conduzido por Li et al. (2025), analisou mais de 456 mil pull requests submetidos por **Agentes Autônomos** em 61 mil repositórios GitHub, revelando que esses agentes já são participantes reais do desenvolvimento de software em produção. Os resultados mostram que agentes como o Codex OpenAI (2025) conseguem fechar pull requests até dez vezes mais rapidamente que humanos, embora disparidades significativas em taxas de aceitação permaneçam quando comparados a contribuições humanas em tarefas complexas de desenvolvimento de funcionalidades e correção de bugs. Esses achados evidenciam tanto o potencial quanto as limitações dos agentes autônomos em contextos reais, justificando a necessidade de estudos comparativos controlados entre os dois paradigmas.

A evolução de ES 2.0 para ES 3.0 não é meramente incremental, mas representa uma mudança qualitativa no locus da agência. Em ES 2.0, o desenvolvedor é o agente principal que utiliza IA como ferramenta; em ES 3.0, sistemas de agentes são os executores principais enquanto o desenvolvedor assume o papel de orquestrador estratégico e revisor. Essa transição implica novas formas de orquestração, novos modelos de governança e novos desafios metodológicos para avaliação científica, particularmente em relação à atribuição de responsabilidade por artefatos gerados, rastreabilidade de decisões e manutenibilidade do código resultante.

2.3 Agentes Inteligentes: Fundamentos Teóricos

A compreensão adequada dos paradigmas ES 2.0 e ES 3.0 exige o domínio dos fundamentos teóricos que definem o que é um agente inteligente. O trabalho seminal de Wooldridge & Jennings (1995), publicado no *The Knowledge Engineering Review*, representa o ponto de partida canônico para essa discussão. Os autores propõem uma definição que distingue entre uma noção "fraca"e uma noção "forte"de agência.

Na noção fraca, um agente é definido como um sistema computacional — hardware ou, mais comumente, software — que exibe as seguintes propriedades fundamentais: **autonomia**, operando sem intervenção direta de humanos e possuindo algum nível de controle sobre suas ações e estado interno; **habilidade social**, interagindo com outros agentes e possivelmente com humanos por meio de alguma linguagem de comunicação entre agentes; **reatividade**, percebendo o ambiente — que pode ser o mundo físico, uma interface gráfica, outros agentes ou a Internet — e respondendo a mudanças nele de forma oportuna; e **proatividade**, não apenas reagindo ao ambiente, mas exibindo comportamento orientado a objetivos por meio de iniciativa própria.

Na noção forte, utilizada especialmente na literatura de Inteligência Artificial, agentes são adicionalmente caracterizados por meio de noções mentalistas, sendo o modelo Belief-Desire-Intention (BDI) o mais amplamente adotado para essa conceptualização. O modelo BDI, desenvolvido por Bratman (1987) e formalizado por Rao & Georgeff (1995), estrutura o raciocínio do agente em três atitudes mentais: **crenças**, representando o conhecimento do agente sobre o ambiente, si mesmo e outros agentes; **desejos**, representando os estados que o agente quer alcançar; e **intenções**, representando a sequência de passos comprometida pelo agente para a realização de um desejo.

Além das propriedades fundantes definidas por Wooldridge & Jennings (1995), os agentes modernos incorporam capacidades adicionais que expandem significativamente seu potencial. **Planejamento** refere-se à capacidade de decom-

por objetivos complexos em sequências de ações executáveis, antecipando dependências e riscos. **Memória** abrange tanto o contexto imediato da tarefa em execução quanto o histórico de interações e conhecimentos de longo prazo que informam decisões futuras. **Aprendizado** permite que agentes refinem seu comportamento com base em experiências passadas, aproximando-se de modelos de reforço e adaptação contínua. **Tomada de decisão** constitui uma capacidade crítica em ambientes dinâmicos e parcialmente observáveis, exigindo que agentes façam escolhas com informação incompleta.

O surgimento dos LLMs transformou substancialmente a realidade prática dos agentes inteligentes. Enquanto os agentes clássicos eram implementados com técnicas simbólicas e requeriam representação explícita do conhecimento do domínio, os LLM-based agents utilizam modelos de linguagem de grande escala como seu "cérebro", integrando-os com ferramentas externas, mecanismos de planejamento e módulos de memória. Essa convergência expandiu dramaticamente o espectro de tarefas que agentes podem executar de forma autônoma, tornando possível que um único agente navegue por repositórios de código, escreva e execute testes, interaja com APIs externas e gere documentação — tudo a partir de uma intenção expressa em linguagem natural.

2.4 Sistemas Multiagentes: Coordenação, Cooperação e Arquiteturas Distribuídas

A limitação de um único agente em lidar com problemas complexos que requerem conhecimentos especializados em múltiplos domínios motivou o desenvolvimento dos Sistemas Multiagentes (SMA). Um SMA é definido como um sistema computacional composto por múltiplos agentes interativos, no qual cada agente possui suas próprias capacidades, objetivos e conhecimentos, e os agentes interagem entre si para resolver problemas que são difíceis ou impossíveis de resolver por um único agente.

As propriedades fundamentais que tornam os SMAs adequados para problemas complexos incluem: **flexibilidade e escalabilidade**, pois podem adaptar-se a ambientes em mudança adicionando, removendo e modificando agentes; **robustez e confiabilidade**, decorrentes da descentralização do controle que permite operação contínua mesmo com a falha de alguns componentes; **auto-organização e coordenação**, onde agentes podem organizar-se com base em regras de comportamento emergente para divisão do trabalho e tomada de decisão coordenada; e operação em tempo real, com respostas situacionais imediatas sem necessidade de supervisão humana.

A convergência entre LLMs e SMA deu origem aos ***LLM-based Multi-Agent Systems***, que representam uma das direções mais promissoras da pesquisa contemporânea em Inteligência Artificial. Nessas arquiteturas, LLMs são utilizados como o "cérebro"de agentes individuais dentro de um sistema maior, trazendo capacidades sofisticadas de raciocínio e linguagem para cada participante. A pesquisa de Tran et al. (2025) sobre mecanismos de colaboração em SMA baseados em LLMs demonstra que esses sistemas superam abordagens de agente único quando os mecanismos de colaboração são adequadamente projetados, embora sistemas com design subótimo possam ser superados por prompts bem elaborados para um único agente.

Os mecanismos de colaboração em sistemas multiagentes baseados em LLMs podem ser classificados em quatro dimensões principais:

1. **Tipologia da colaboração**: caracteriza a natureza da interação entre os agentes, podendo ocorrer na forma de *cooperação*, quando os agentes trabalham em direção a objetivos compartilhados; *competição*, quando perseguem objetivos conflitantes; ou *cooperetição*, combinando aspectos colaborativos e competitivos.

2. **Estrutura de comunicação**: define como as informações são compartilhadas entre os agentes, podendo ser *centralizada*, quando um agente coordenador gerencia as interações; *descentralizada*, quando a comunicação ocorre diretamente entre os agentes; ou *hierárquica*, quando os agentes são organizados em diferentes níveis de responsabilidade.

3. **Estratégia de coordenação**: determina como as ações dos agentes são sincronizadas ao longo da execução, podendo ser *estática*, baseada em regras previamente definidas e conhecimento de domínio, ou *dinâmica*, adaptando-se às mudanças do ambiente e às necessidades da tarefa.

4. **Orquestração**: estabelece como os agentes são organizados para colaborar, definindo os canais de comunicação, a sequência das interações, os fluxos de execução e as regras que governam o comportamento coletivo do sistema.

Essas quatro dimensões constituem os principais elementos arquiteturais dos sistemas multiagentes baseados em LLMs e influenciam diretamente sua eficiência, escalabilidade e capacidade de resolver tarefas complexas.

Um aspecto central para a aplicação de SMA à Engenharia de Software é a especialização de agentes. Trabalhos como Hong et al. (2023) e Qian et al. (2023) demonstraram que a atribuição de papéis distintos — gerente de produto, arquiteto, engenheiro, QA — a instâncias diferentes de LLMs melhora a qualidade dos artefatos em comparação com abordagens de agente único. Essa especialização encoraja modularidade, aumenta a reusabilidade de módulos individuais e melhora o desempenho global do sistema por meio de divisão explícita do trabalho. O estudo de Soorati et al. (2023) sobre sistemas multiagentes confiáveis acrescenta à discussão a dimensão da confiabilidade: à medida que SMA assumem tarefas com maior autonomia, mecanismos de raciocínio sobre responsabilidade, delegação e confiança tornam-se indispensáveis para garantir operação segura e auditável.

2.5 Orquestração de Agentes: Grafos, Estado e Workflows Complexos

A crescente adoção de arquiteturas multiagentes para resolução de problemas complexos tornou evidente a necessidade de mecanismos robustos de orquestração — o conjunto de processos que governa como agentes são criados, ativados, coordenados e monitorados em um sistema. A orquestração vai além da simples composição de agentes: ela engloba a definição de workflows, o gerenciamento de estado compartilhado, o roteamento dinâmico de tarefas e a garantia de observabilidade sobre o processo de execução.

Arquiteturas de orquestração modernas frequentemente representam o fluxo de execução como grafos direcionados, nos quais nós representam agentes ou funções executáveis e arestas representam dependências e fluxos de controle. Essa abordagem, exemplificada por frameworks como *LangGraph* (LangChain, Inc. (2025a)), permite a execução tanto sequencial quanto paralela de agentes, suporta ciclos condicionais necessários para processos iterativos como revisão e refinamento, e oferece primitivas para persistência de estado entre as etapas do workflow. A representação grafo-orientada do fluxo de execução também facilita a identificação visual de dependências entre agentes e a rastreabilidade das decisões ao longo do pipeline.

O gerenciamento de contexto em arquiteturas multiagentes constitui um dos problemas mais significativos da área. A arquitetura *CodeAir*, proposta por Rangel (2025), endereça esse problema por meio de uma estratégia de limpeza e reinjeção de contexto entre fases: cada fase inicia com uma sessão limpa, mas contexto relevante é explicitamente reinjetado por meio de prompts estruturados e artefatos externos persistidos em git. Essa abordagem trata o contexto como um recurso renovável em vez de uma restrição a ser gerenciada, privilegiando reprodutibilidade e consistência sobre conveniência conversacional. A estratégia demonstrou resultados expressivos em produção — redução de 96% no tempo de desenvolvimento de funcionalidades complexas —, corroborando a tese de que a orquestração estruturada supera a interação direta com LLMs para tarefas profissionais de desenvolvimento.

Além do gerenciamento de contexto, a observabilidade constitui uma dimensão crítica em arquiteturas de orquestração. Em sistemas complexos compostos por múltiplos agentes, a capacidade de rastrear cada decisão — qual agente executou qual ação, com quais entradas, gerando quais saídas, em quanto tempo e com qual custo computacional — é fundamental tanto para depuração quanto para análise científica do comportamento do sistema. A instrumentação com ferramentas como *LangSmith* (LangChain, Inc. (2025b)) permite a coleta sistemática dessas métricas, fornecendo traços de execução detalhados que possibilitam análise de gargalos, cálculo de custo por tarefa e avaliação da distribuição de responsabilidade entre agentes.

2.6 Capacidades Fundamentais de Agentes Baseados em Large Language Models

A implementação prática de agentes baseados em Large Language Models requer a integração de diferentes capacidades que ampliam o potencial do modelo de linguagem, permitindo-lhe interagir com o ambiente, raciocinar sobre problemas complexos, manter contexto ao longo do tempo e acessar informações externas de forma dinâmica. Essas capacidades constituem a base dos agentes modernos utilizados em aplicações de Engenharia de Software.

2.6.1 Utilização de Ferramentas

A utilização de ferramentas (*Tool Calling* ou *Function Calling*) permite que agentes invoquem recursos externos, como APIs, interpretadores de código, mecanismos de busca e sistemas de arquivos. Essa capacidade transforma o LLM em um agente capaz de executar ações no ambiente, verificar resultados e utilizar essas informações como parte do seu processo de tomada de decisão.

2.6.2 Planejamento

O Planejamento é a capacidade do agente de decompor objetivos de alto nível em sequências de ações executáveis. Abordagens como *Chain-of-Thought Prompting*, *Tree of Thoughts* e *ReAct* estruturam o raciocínio do agente em passos explícitos, melhorando tanto a qualidade do planejamento quanto a interpretabilidade do processo decisório. Em arquiteturas multiagentes, o planejamento frequentemente ocorre em múltiplos níveis: um agente planejador de alto nível decompõe o objetivo em subproblemas que são delegados a agentes executores especializados.

2.6.3 Memória

A memória em sistemas de agentes pode ser estruturada em diferentes camadas. A memória de curto prazo é tipicamente gerenciada pela janela de contexto do LLM, enquanto a memória de longo prazo requer mecanismos externos de persistência, como bancos de dados vetoriais que armazenam representações semânticas e informações relevantes para recuperação posterior. A pesquisa de Shang et al. (2024) sobre sistemas de memória AI-native propõe arquiteturas capazes de armazenar e organizar conhecimento persistente, atualizando-o dinamicamente com base no raciocínio do agente e distinguindo entre memória de curto e longo prazo, permitindo a recuperação contextual de informações ao longo da execução.

2.6.4 RAG

O *Retrieval-Augmented Generation (RAG)* estende a capacidade do agente ao permitir que ele recupere dinamicamente informações relevantes de bases de conhecimento externas antes de gerar uma resposta ou tomar uma decisão. Em contextos de desenvolvimento de software, RAG pode ser utilizado para injetar no contexto do agente documentação relevante, exemplos de código similar, padrões arquiteturais estabelecidos e histórico de decisões técnicas. A técnica mitiga limitações dos LLMs em relação ao conhecimento desatualizado e à incapacidade de raciocinar sobre informações específicas de um domínio ou organização.

2.6.5 MCP

O *Model Context Protocol (MCP)*, proposto pela Anthropic (2024), representa um avanço importante na padronização da comunicação entre agentes e ferramentas externas. O protocolo define uma interface unificada para que agentes acessem recursos, invoquem ferramentas e troquem informações com servidores externos, reduzindo a complexidade de integração em arquiteturas multiagentes heterogêneas. A adoção crescente do MCP sugere uma tendência de padronização das interfaces de comunicação em ecossistemas de agentes, análoga ao papel que APIs REST desempenhou na integração de serviços web.

A síntese dessas capacidades constitui o núcleo técnico que diferencia os LLM agents dos sistemas de assistência copiloto que caracterizam a ES 2.0. Enquanto um copiloto res-

ponde a solicitações isoladas dentro de uma janela de contexto limitada, um agente de ES 3.0 executa workflows completos, mantém consciência do estado global do projeto e refina iterativamente seus artefatos por meio de ciclos de planejamento, execução e reflexão. É essa diferença qualitativa que motiva a investigação científica sobre os impactos comparativos dos dois paradigmas sobre o processo de desenvolvimento de software.

2.7 Programação Baseada em Agentes

A programação de sistemas multiagentes possui uma história intelectual rica que precede o advento dos LLMs. A revisão sistemática conduzida por Cardoso & Ferrando (2021) cataloga o estado da arte em linguagens de programação orientadas a agentes (APLs), documentando a evolução desde o trabalho seminal de Shoham (1993) sobre *Agent-Oriented Programming* até as plataformas modernas. O modelo BDI emerge como o mais amplamente adotado nas APLs catalogadas, sendo utilizado em sete das quinze linguagens identificadas na revisão, e plataformas como Boissier et al. (2013), Bordini et al. (2007) e Bellifemine et al. (2001) consolidaram-se como referências para o desenvolvimento de Sistemas Multiagentes em contextos acadêmicos e industriais.

Um dos maiores desafios históricos no desenvolvimento de sistemas multiagentes tem sido a lacuna cognitiva entre os conceitos das linguagens orientadas a agentes e os paradigmas mainstream de programação. Cardoso & Ferrando (2021) argumentam que a falta de incentivo para desenvolvedores adotarem APLs — dado que comportamentos agentivos simples podem ser implementados em linguagens convencionais com sobrecarga mínima — é um dos principais obstáculos à adoção ampla de programação baseada em agentes. A chegada dos LLMs representa uma solução parcial para esse problema: ao utilizar um modelo de linguagem como núcleo do agente, desenvolvedores podem especificar comportamentos agentivos em linguagem natural sem precisar dominar linguagens formais especializadas.

O surgimento de frameworks como LangChain, AutoGen e CrewAI representa uma nova geração de plataformas para desenvolvimento de SMA baseados em LLMs, que abstraem a complexidade técnica da integração entre modelos, ferramentas e memória, oferecendo primitivas de alto nível para composição de workflows agentivos. LangGraph, em particular, introduz uma primitiva de grafo direcionado para representar workflows de agentes, com suporte nativo a estado compartilhado, execução condicional e ciclos iterativos. Essa representação grafo-orientada alinha-se com os padrões arquiteturais identificados na literatura de SMA e constitui a base tecnológica sobre a qual a framework **Uranium**, artefato desta pesquisa, será construída

2.8 Segurança, Confiabilidade e Governança em Sistemas de Agentes

À medida que sistemas de agentes autônomos avançam de ambientes controlados para uso em produção, as dimensões de segurança, confiabilidade e governança assumem importância crescente. O trabalho de Soorati et al. (2023) sobre sistemas multiagentes confiáveis e centrados no humano documenta duas décadas de pesquisa sobre como projetar SMA que operem de forma segura e coordenada, reconhecendo e compensando falhas e trabalhando em parceria com humanos. Os autores distinguem entre confiança — a disposição do usuário em depender do sistema — e confiabilidade — a capacidade demonstrável do sistema de operar conforme especificado —, argumentando que ambas devem ser consideradas no design de sistemas autônomos.

A crescente autonomia dos agentes torna indispensável a adoção de mecanismos de segurança e governança capazes de garantir que suas ações permaneçam alinhadas aos objetivos do sistema. Entre os principais desafios destacam-se ataques de *prompt injection*, comprometimento do contexto utilizado durante o raciocínio e propagação de erros em workflows colaborativos. Em resposta, arquiteturas contemporâneas têm incorporado políticas de autorização, controle de acesso a ferramentas, monitoramento contínuo e rastreabilidade das ações executadas, promovendo maior confiabilidade, transparência e auditabilidade.

Embora ganhos de produtividade sejam frequentemente utilizados para avaliar sistemas baseados em agentes, eles não são suficientes para caracterizar a qualidade do desenvolvimento. Trabalhos recentes sobre o impacto da Inteligência Artificial na Engenharia de Software, como o de Borg et al. (2023), ressaltam a importância de complementar métricas de desempenho com indicadores relacionados à qualidade e à manutenibilidade do código produzido. Assim, neste trabalho, a comparação entre os paradigmas de Engenharia de Software 2.0 e Engenharia de Software 3.0 considera não apenas aspectos de produtividade, mas também métricas como *Maintainability Index*, *Cyclomatic Complexity* e *Mutation Score*, permitindo uma avaliação mais abrangente dos artefatos gerados.

3 Desenvolvimento

Essa seção, as vezes denominada Materiais e Métodos, contém a contribuição pessoal do autor. Por isso, o ideal é evitar citar referências, pois considera-se que todos os conceitos já foram dados no Referencial Teórico. O objetivo é descrever em detalhes como foi conduzido o estudo para solução do problema, incluindo artefatos ou produtos que foram gerados (ex. modelos, software, pesquisa, documentação etc).

É importante fornecer informações que permitam ao leitor avaliar a adequação dos métodos que utilizou, bem como proporcionar confiabilidade e validade aos resultados obtidos. Essa seção deverá conter informações que possibilitem a reprodução do estudo. **Sugestões de conteúdo para o Desenvolvimento**:

- Descrição do tipo de pesquisa como: Abordagem (Qualitativa, Quantitativa?); Natureza (Teórica, Prática); Procedimento (ex. Pesquisa Experimental, Survey, Estudo de Caso etc) e Método de pesquisa escolhido para conduzir o desenvolvimento do TCC;
- Dados utilizados (Materiais): São de que tipo? Como foram obtidos? Qual a Quantidade? Onde obtê-los?
- Evidências do que foi desenvolvido (ex. Requisitos, modelos, diagramas, figuras, algoritmos);
- Projeto de Experimentos: Como foi testado? Qual é o planejamento dos experimentos? Que métricas e métodos

estatísticos utilizará para verificar se os objetivos foram atendidos ou não? Quais são os fatores, os níveis e variáveis de resposta dos experimentos? Explique de modo a possibilitar a reprodução;

- Descrição do Ambiente de Desenvolvimento e de Experimentos (ex. hardware, softwares, sistemas operacional, bibliotecas. Incluir número de versões). Deixar claro se os parâmetros dos métodos utilizados estão conforme padrão ou se foram alterados (informar os valores dos parâmetros quando for o caso).

Para mais informações sobre este capítulo acesse `www.youtube.com/watch?v=fVmmPZsmtbE`.

3.1 Exemplo de Algoritmo

O comando `\begin{algorithm}` fornece uma estrutura para escrita de algoritmos em pseudocódigo, como por exemplo o Algoritmo 1.

Algoritmo 1: *PieceRank*

Dados: G
1 $\pi_{PE} = [\,]$; $PC = [\,]$;
2 $rankPecas \leftarrow$ `constroiRankPecas`$(vetCent)$;
3 $\pi_{PE}, PC \leftarrow peca_{ref}$;
4 $rankPecas$.`atualizaPagerankAcumulado`$(adjtsPeca)$;
5 **enquanto** $rankPecas \neq \emptyset$ **faça**
6 $adjtsPeca_{ref} \leftarrow$ `ordenaPagerank`$(adjtsPeca_{ref})$;
7 **para** $adjtPeca \in adjtsPeca_{ref}$ **faça**
8 $\pi_{PE}, PC \leftarrow adjtPeca$;
9 $rankPecas$.`atualizaPagerank`$(adjtsAdjtPeca)$;
10 **fim**
11 $PC \leftarrow PC - peca_{ref}$;
12 $peca_{ref} \leftarrow PC[0]$;
13 $adjtsPeca_{ref} \leftarrow$ `obtemAdjacentes`$(peca_{ref}) - \pi_{PE}$;
14 **fim**
15 **retorna** π_{PE}

Todos os Algoritmos incluídos no documento precisam ser descritos no texto, assim como todas as tabelas e figuras. Utilize os números das linhas para descrever o que ocorre no algoritmo (ex. Entre as linhas 7 e 10 ...).

Mais informações sobre o pacote `algorithm2e` estão disponíveis em `https://edisciplinas.usp.br/pluginfile.php/5306173/mod_resource/content/1/algoritmo_exemplo.pdf`.

3.1.1 Página em Paisagem

Caso necessite inserir tabelas ou figuras muito largas utilize `\paisagem{x}{...}` para obter uma página no formato paisagem, conforme exemplo da Figura 1. O parâmetro `x` recebe os valores `f` ou `t`, para figuras ou tabelas, respectivamente. A sintaxe do comando fica:

```
\paisagem{f}{
    \includegraphics...
}
```

3.2 Fórmulas Matemáticas

Consulte o link `https://en.wikibooks.org/wiki/LaTeX/Mathematics` que contém diversos conteúdos sobre a construção de fórmulas em LaTeX, além de símbolos matemáticos. A Equação 1 contém um exemplo:

$$r_{k+1}(P_i) = \sum_{P_j \in B_{P_i}} \frac{r_k(P_j)}{|P_j|} \tag{1}$$

A Equação 2 é um outro exemplo (OBS. Todas as variáveis de uma fórmula precisam ser descritas no texto):

$$f(n) = \begin{cases} n/2 & \text{se } n \text{ é ímpar} \\ -(n+1)/2 & \text{se } n \text{ é par} \end{cases} \tag{2}$$

3.3 Códigos-fonte

Embora um algoritmo, como mostrado na Seção 3.1, seja uma das maneiras mais elegantes de se apresentar uma implementação de software, existem algumas situações em que pode ser necessário incluir um código-fonte (ou parte dele) no texto.

Esta ação pode ser realizada através do comando `\lstinputlisting`. De modo análogo às figuras e tabelas, todos os códigos também são numerados e devem estar citados no texto. Por exemplo: o programa ilustrado pelo Código 1 foi inserido no documento através do seguinte comando:

```
\lstinputlisting[language=Python,
caption={Implementação do
algoritmo Bubble Sort},
label=bubblePy]{./elemtextuais/bubble.py}
```

Código 1: Implementação do algoritmo Bubble Sort

```
1  # Codigo Python que implementa o algoritmo
       Bubble Sort
2
3  def bubbleSort(arr):
4    n = len(arr)
5
6    for i in range(n):
7      for j in range(0, n-i-1):
8        if arr[j] > arr[j+1] :
9          arr[j], arr[j+1] = arr[j+1], arr[j]
10
11 arr = [64, 34, 25, 12, 22, 11, 90]
12
13 bubbleSort(arr)
14
15 print ("Sorted array is:")
16 for i in range(len(arr)):
17   print ("%d" %arr[i]),
```

Para mais detalhes sobre a inclusão de códigos-fonte, consulte: `https://en.wikibooks.org/wiki/LaTeX/Source_Code_Listings`.

4 Resultados

Nesta seção são dadas as evidências dos experimentos realizados para validar a solução que propôs na Seção Desenvolvimento. Os **Resultados** correspondem aos fatos verificados com o estudo, enquanto a **Discussão** gira em torno dos aspectos a serem interpretados sobre esses fatos. Os resultados

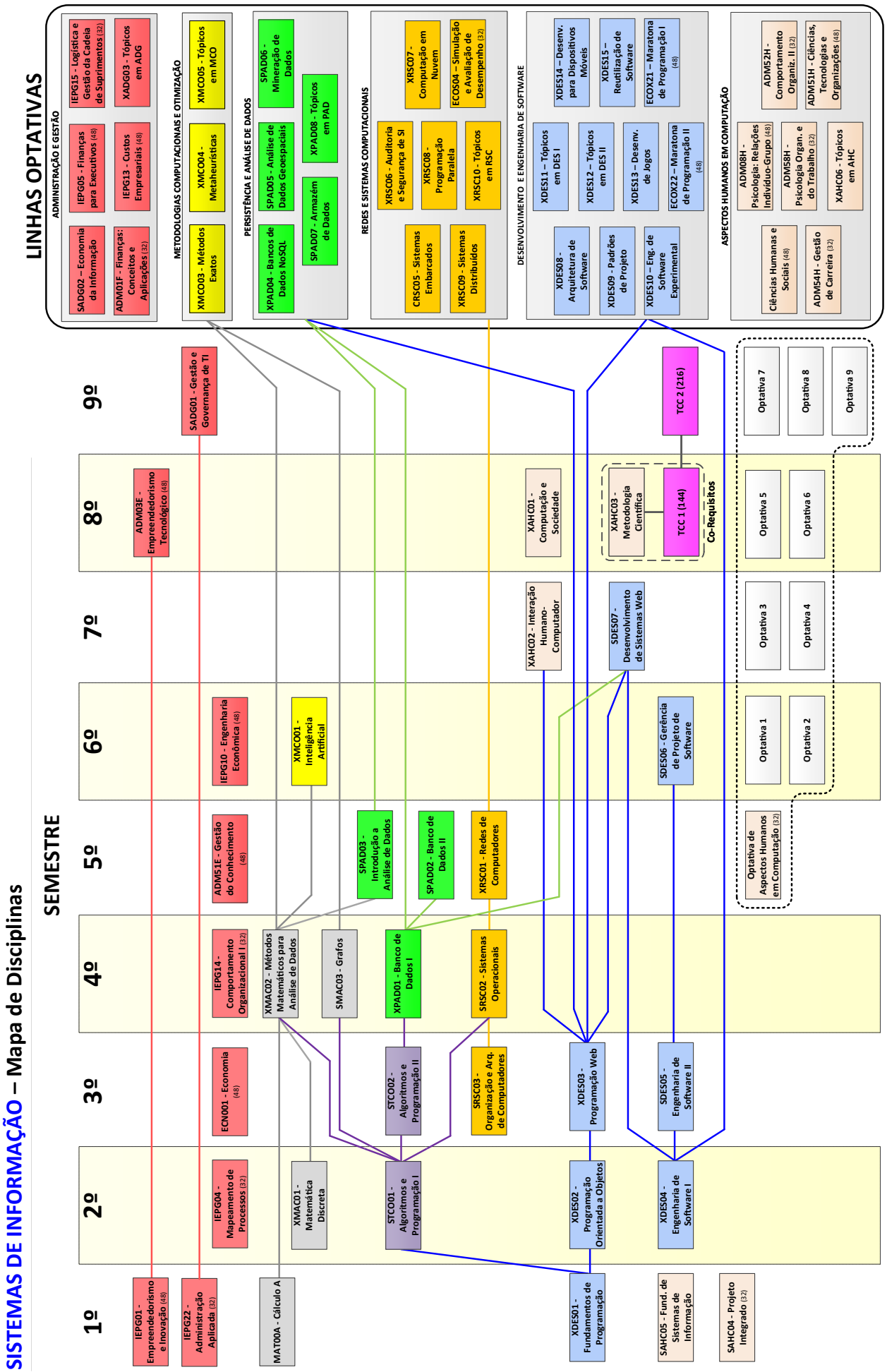


Figura 1: Exemplo de utilização de uma página configurada como paisagem para figuras ou tabelas muito grandes.

devem ser devidamente apresentados, analisados e só então discutidos. Em alguns casos a discussão dos resultados é feita na Seção Conclusão. **Dicas para escrita dos resultados**:

- Apresentar os resultados de cada planejamento de experimentos que foi definido. Para isso, além da descrição textual dos resultados, o uso de tabelas, figuras e gráficos permite uma melhor visualização;
- Procure ter uma linguagem neutra. Valorize o que deu certo e mostre o que deu errado, seja imparcial. Apresente os pontos positivos e negativos;
- Faça conjecturas que forneçam uma provável explicação para justificar os resultados;
- A partir das evidências deve-se:
  - Verificar se os objetivos foram atendidos; e
  - Descrever se a hipótese foi confirmada ou não.

Alguns autores optam por fazer as discussões na própria Seção Resultados, mas geralmente utilizando uma subseção específica para isso. Uma alternativa é fazer a discussão na Seção Conclusões. **Dicas sobre o que discutir:**

- Relacionamentos entre os fatos e resultados observados;
- Princípios, relações, generalizações observadas nos Experimentos;
- Exceção ou falta de relação, pontos incertos;
- Mostrar que resultados e interpretações concordam (ou contrastam) com trabalhos previamente publicados; e
- Implicações teóricas e possíveis aplicações práticas.

Consulte os links a seguir com dicas úteis para esta seção:

- "Como fazer resultados e discussões no TCC" https://viacarreira.com/resultados-e-discussoes-no-tcc.
- "Como fazer a análise sobre os resultados coletados para o TCC" www.icguedes.pro.br/como-fazer-analise-sobre-os-resultados-coletados-para-tcc.

5 Conclusões

Nesta seção é feito o fechamento do trabalho, devendo ser descrito de forma sintética o problema abordado, a solução desenvolvida, como a solução foi validada, e quais os resultados e constatações mais significativas. No geral, a conclusão pode ser dividida em três partes: A (*i*) conclusão (hipóteses foram comprovadas? qual foi o melhor método? etc), (*ii*) contribuições e os (*iii*) trabalhos futuros.

Uma dica é citar os objetivos do trabalho (geral e específicos) e descrever como fez para que fossem alcançados (ou não). Destaque a principal contribuição do trabalho, os pontos positivos e os negativos, associe evidências para cada conclusão que apresentar.

Nos **Trabalhos Futuros** é importante dar ao leitor uma noção das possibilidades de pesquisa que podem ser desenvolvidas a partir do que foi feito no seu trabalho. Isso aumenta as possibilidades do seu trabalho ser citado por outros autores.

Em relação as **Referências Bibliográficas**, o ideal é organizar os artigos que baixar por "*ano, Sobrenome1°autor - Título*" de modo a evitar duplicidades. A ferramenta Mendeley www.mendeley.com é útil para organização de arquivos de referências (local ou nuvem), localização de conteúdo e para obtenção de referências no formato LaTeX.

Inclua as referências bibliográficas no arquivo "***referencias.bib***". Referências em LaTeX podem ser obtidas pelo Google Acadêmico conforme passos descritos na Figura 2. Atente-se pois muitas vezes as referências obtidas pelo Google vêm incompletas.

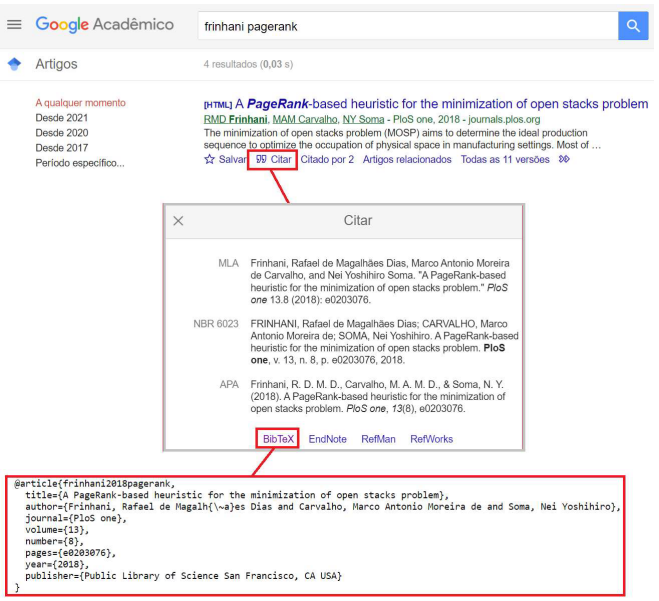


Figura 2: Passos para obter referências LaTeX pelo Google Acadêmico.

Para mais informações sobre os campos exigidos em cada tipo de referência em LaTeXacesse: http://paginapessoal.utfpr.edu.br/jamhour/publicacoes/arquivos/00_Compilado_JabRef_dez2015.pdf

Agradecimentos

Seção reservada para agradecimentos a pessoas, instituições ou empresas que viabilizaram ou auxiliaram de forma expressiva o desenvolvimento do trabalho. Neste formato de artigo, esta seção deve ter no máximo um parágrafo e ser bem sucinta (até 4 linhas). Edite o arquivo "*elemtextuais > agradecimentos.tex*".

Referências

Anthropic (2024). Introducing the model context protocol. https://www.anthropic.com/news/model-context-protocol. Acessado em: 03 jul. 2026.

Anthropic (2025). Claude code. https://anthropic.com. [Online; accessed 06-May-2026].

Anysphere (2022). Cursor. https://cursor.com. [Online; accessed 06-May-2026].

Bellifemine, F. L., Poggi, A., & Rimassa, G. (2001). Jade: A fipa-compliant agent framework. *Proceedings of PAAM 2001*, 99, 97–108.

Boissier, O., Bordini, R. H., Hübner, J. F., & Ricci, A. (2013). Multi-agent oriented programming with jacamo. *Science of Computer Programming*, 78(6), 747–761.

Bordini, R. H., Hübner, J. F., & Wooldridge, M. (2007). Programming multi-agent systems in agentspeak using jason. *John Wiley & Sons*.

Borg, M., Taibi, D., Lenarduzzi, V., & outros (2023). Generative artificial intelligence for software engineering: A research agenda. *arXiv preprint*. [Online; accessed 03-Jul-2026].

Bratman, M. E. (1987). *Intention, Plans, and Practical Reason*. Harvard University Press.

Cardoso, R. C. & Ferrando, A. (2021). A review of agent-based programming for multi-agent systems. *Computers*, 10(2), 16.

GitHub (2021). Github copilot: Your ai pair programmer. https://github.com/features/copilot. [Online; accessed 06-May-2026].

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2023). Metagpt: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.

LangChain, Inc. (2025a). Langgraph documentation. https://langchain-ai.github.io/langgraph/. Acessado em: 03 jul. 2026.

LangChain, Inc. (2025b). Langsmith documentation. https://docs.smith.langchain.com/. Acessado em: 03 jul. 2026.

Li, H., Zhang, H., & Hassan, A. E. (2025). The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*.

OpenAI (2025). Introducing codex.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2023). Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*.

Rangel, G. A. (2025). Codeair: A multi-agent orchestration framework for ai-assisted software development. *SSRN Electronic Journal*.

Rao, A. S. & Georgeff, M. P. (1995). Bdi agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, (pp. 312–319).

Shang, J., Zheng, Z., Wei, J., Ying, X., Tao, F., & Team, M. (2024). Ai-native memory: A pathway from llms towards agi. *arXiv preprint arXiv:2406.18312*.

Shoham, Y. (1993). Agent-oriented programming. *Artificial Intelligence*, 60(1), 51–92.

Soorati, M. D., Gerding, E. H., Marchioni, E., Naumov, P., Norman, T. J., Ramchurn, S. D., Rastegari, B., Sobey, A., Stein, S., Tarpore, D., Yazdanpanah, V., & Zhang, J. (2023). From intelligent agents to trustworthy human-centred multiagent systems. *AI Communications*, 36(4), 311–327.

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., & Nguyen, H. D. (2025). Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*.

Wooldridge, M. & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2), 115–152.

